

# Practica 6 - Funciones Verosimilitud

Juan Elias Lara Soto

```
library("testthat")
```

Warning: package 'testthat' was built under R version 4.4.3

---

---

```
#' Log-verosimilitud para distribución Exponencial
#'  
#' @param lambda Tasa de la distribución (>0)  
#' @param x Vector de observaciones  
#' @return Valor numérico de la log-verosimilitud  
#' @examples  
#' x <- rexp(30, rate = 1)  
#' logLik_exp(lambda = 1, x)  
logLik_exp <- function(lambda, x) {  
  n <- length(x)  
  n*log(lambda) - lambda*sum(x)  
}
```

```
test_that("logLik_exp calcula un número finito", {  
  x <- rexp(30, 1)  
  expect_true(is.finite(logLik_exp(1, x)))  
})
```

Test passed

```

#' Log-verosimilitud para distribución Normal
#'
#' @param mu Media de la distribución
#' @param sigma Desviación estándar (>0)
#' @param x Vector de observaciones
#' @return Valor numérico de la log-verosimilitud
#' @examples
#' x <- rnorm(30, mean = 10, sd = 3)
#' logLik_normal(mu = 10, sigma = 3, x)
logLik_normal <- function(mu, sigma, x) {
  n <- length(x)
  -n/2*log(2*pi*sigma^2) - 1/(2*sigma^2)*sum((x-mu)^2)
}

```

```

test_that("logLik_normal calcula un número finito", {
  x <- rnorm(30, 10, 3)
  expect_true(is.finite(logLik_normal(10, 3, x)))
})

```

Test passed

```

#' Log-verosimilitud para distribución Uniforme
#'
#' @param a Límite inferior
#' @param b Límite superior
#' @param x Vector de observaciones
#' @return Log-verosimilitud, -Inf si x fuera de [a,b]
#' @examples
#' x <- runif(30, min = 2, max = 9)
#' logLik_unif(a = 2, b = 9, x)
logLik_unif <- function(a, b, x) {
  n <- length(x)
  if(any(x < a | x > b)) return(-Inf)
  n*log(1/(b-a))
}

```

```

test_that("logLik_unif calcula correctamente", {
  x <- runif(30, 2, 9)
  expect_true(is.finite(logLik_unif(2, 9, x)))
})

```

Test passed

```
#' Log-verosimilitud para distribución Bernoulli
#'
#' @param p Probabilidad de éxito (0<p<1)
#' @param x Vector de 0s y 1s
#' @return Valor de la log-verosimilitud
#' @examples
#' x <- rbinom(30, 1, 0.7)
#' logLik_ber(p = 0.7, x)
logLik_ber <- function(p, x) {
  n <- length(x)
  sum(x)*log(p) + (n-sum(x))*log(1-p)
}
```

```
test_that("logLik_ber calcula correctamente", {
  x <- rbinom(30, 1, 0.7)
  expect_true(is.finite(logLik_ber(0.7, x)))
})
```

Test passed

```
#' Log-verosimilitud para distribución Binomial
#'
#' @param n1 Número de ensayos
#' @param p Probabilidad de éxito
#' @param x Vector de éxitos
#' @return Log-verosimilitud
#' @examples
#' x <- rbinom(30, 10, 0.3)
#' logLik_bin(n1 = 10, p = 0.3, x)
logLik_bin <- function(n1, p, x) {
  n2 <- length(x)
  sum(lchoose(n1, x) + x*log(p) + (n1-x)*log(1-p))
}
```

```
test_that("logLik_bin calcula correctamente", {
  x <- rbinom(30, 10, 0.3)
  expect_true(is.finite(logLik_bin(10, 0.3, x)))
})
```

Test passed

```

#' Log-verosimilitud para distribución Geométrica
#'
#' @param p Probabilidad de éxito
#' @param x Vector de conteos de ensayos hasta el primer éxito
#' @return Log-verosimilitud
#' @examples
#' x <- rgeom(30, 0.3) + 1
#' logLik_geom(p = 0.3, x)
logLik_geom <- function(p, x) {
  n <- length(x)
  n*log(p) + sum(x-1)*log(1-p)
}

```

```

test_that("logLik_geom calcula correctamente", {
  x <- rgeom(30, 0.3) + 1
  expect_true(is.finite(logLik_geom(0.3, x)))
})

```

Test passed

```

#' Log-verosimilitud para distribución Poisson
#'
#' @param lambda Media de la distribución (>0)
#' @param x Vector de conteos
#' @return Log-verosimilitud
#' @examples
#' x <- rpois(30, 3)
#' logLik_poiss(lambda = 3, x)
logLik_poiss <- function(lambda, x) {
  n <- length(x)
  -n*lambda + sum(x)*log(lambda) - sum(lfactorial(x))
}

```

```

test_that("logLik_poiss calcula correctamente", {
  x <- rpois(30, 3)
  expect_true(is.finite(logLik_poiss(3, x)))
})

```

Test passed

```

#' Log-verosimilitud para distribución Gamma
#'
#' @param alpha Parámetro de forma (>0)
#' @param beta Parámetro de tasa (>0)
#' @param x Vector de observaciones
#' @return Log-verosimilitud
#' @examples
#' x <- rgamma(30, shape = 2, rate = 1)
#' logLik_gamma(alpha = 2, beta = 1, x)
logLik_gamma <- function(alpha, beta, x) {
  n <- length(x)
  n*(alpha*log(beta) - lgamma(alpha)) + (alpha-1)*sum(log(x)) - beta*sum(x)
}

```

```

test_that("logLik_gamma calcula correctamente", {
  x <- rgamma(30, 2, 1)
  expect_true(is.finite(logLik_gamma(2, 1, x)))
})

```

Test passed

```

#' Log-verosimilitud para distribución Beta
#'
#' @param alpha Parámetro de forma 1 (>0)
#' @param beta Parámetro de forma 2 (>0)
#' @param x Vector de observaciones entre 0 y 1
#' @return Log-verosimilitud
#' @examples
#' x <- rbeta(30, 2, 5)
#' logLik_beta(alpha = 2, beta = 5, x)
logLik_beta <- function(alpha, beta, x) {
  n <- length(x)
  if(any(x < 0 | x > 1)) return(-Inf)
  n*(lgamma(alpha+beta) - lgamma(alpha) - lgamma(beta)) + (alpha-1)*sum(log(x)) + (beta-1)*sum(1-x)
}

```

```

test_that("logLik_beta calcula correctamente", {
  x <- rbeta(30, 2, 5)
  expect_true(is.finite(logLik_beta(2, 5, x)))
})

```

Test passed

```
#' Log-verosimilitud para distribución Chi-cuadrado
#'
#' @param k Grados de libertad (>0)
#' @param x Vector de observaciones
#' @return Log-verosimilitud
#' @examples
#' x <- rchisq(30, df = 5)
#' logLik_chi2(k = 5, x)
logLik_chi2 <- function(k, x) {
  n <- length(x)
  n*(-k/2*log(2) - lgamma(k/2)) + (k/2 - 1)*sum(log(x)) - sum(x)/2
}

test_that("logLik_chi2 calcula correctamente", {
  x <- rchisq(30, 5)
  expect_true(is.finite(logLik_chi2(5, x)))
})
```

Test passed

```
n <- 30
mu <- 10
sigma <- 3
x <- rnorm(n, mean = mu, sd = sigma)
logLik_normal(mu = mu, sigma = sigma, x) # tiene que ser mayor
```

```
[1] -70.55316
```

```
logLik_exp(lambda = mu, x)
```

```
[1] -2976.859
```

```
n <- 30
mu <- 10
sigma <- 3
lambda <- 10
x <- rexp(n, rate = lambda)
logLik_normal(mu = mu, sigma = sigma, x)
```

```
[1] -223.7734
```

```
logLik_exp(lambda = mu, x) # tiene que ser mayor
```

```
[1] 37.95415
```

```
# Simulamos datos normales
n <- 30
mu <- 10
sigma <- 3
x <- rnorm(n, mean = mu, sd = sigma)

# Calculamos logLik en varias distribuciones
logLik_normal(mu = mu, sigma = sigma, x) # tiene que ser mayor
```

```
[1] -74.38113
```

```
logLik_exp(lambda = mu, x)
```

```
[1] -2958.126
```

```
logLik_unif(a = min(x), b = max(x), x)
```

```
[1] -77.72701
```

```
logLik_pois(lambda = mean(x), x)
```

```
[1] -73.85819
```

```
# Simulamos datos exponenciales
n <- 30
lambda <- 0.5
x <- rexp(n, rate = lambda)

# Calculamos logLik
logLik_exp(lambda = lambda, x) # tiene que ser mayor
```

```
[1] -55.93388
```

```
logLik_normal(mu = mean(x), sigma = sd(x), x)
```

```
[1] -64.83539
```

```
logLik_unif(a = min(x), b = max(x), x)
```

```
[1] -58.36209
```

```
logLik_pois(lambda = mean(x), x)
```

```
[1] -63.1029
```

```
# Simulamos datos Poisson
n <- 30
lambda <- 3
x <- rpois(n, lambda)

# Calculamos logLik
logLik_pois(lambda = lambda, x) # tiene que ser mayor
```

```
[1] -65.39639
```

```
logLik_normal(mu = mean(x), sigma = sd(x), x)
```

```
[1] -62.70659
```

```
logLik_bin(n1 = max(x)+5, p = mean(x)/(max(x)+5), x)
```

```
[1] -63.43886
```

```
# Simulamos datos uniformes
n <- 30
a <- 0
b <- 5
x <- runif(n, min = a, max = b)

# Calculamos logLik
logLik_unif(a = a, b = b, x) # tiene que ser mayor
```



```
[1] -48.28314
```

```
logLik_normal(mu = mean(x), sigma = sd(x), x)
```

```
[1] -53.55626
```

```
logLik_exp(lambda = 1/mean(x), x)
```

```
[1] -56.25002
```

```
# Simulamos datos Bernoulli
n <- 30
p <- 0.7
x <- rbinom(n, size = 1, prob = p)

# Calculamos logLik
logLik_ber(p = p, x) # tiene que ser mayor
```

```
[1] -16.63133
```

```
logLik_bin(n1 = 1, p = p, x) # en Bernoulli n1=1
```

```
[1] -16.63133
```

```
logLik_pois(lambda = mean(x), x)
```

```
[1] -29.11117
```